

Modifying settings.py for Platform.sh

Now we need to add a section at the end of *settings.py* to modify some settings for the Platform.sh environment. Add this code to the very end of *settings.py*:

```
settings.py  --snip--
# Platform.sh settings.
❶ from platformshconfig import Config

    config = Config()
❷ if config.is_valid_platform():
❸     ALLOWED_HOSTS.append('.platformsh.site')

❹     if config.appDir:
        STATIC_ROOT = Path(config.appDir) / 'static'
❺     if config.projectEntropy:
        SECRET_KEY = config.projectEntropy

    if not config.in_build():
❻         db_settings = config.credentials('database')
        DATABASES = {
            'default': {
                'ENGINE': 'django.db.backends.postgresql',
                'NAME': db_settings['path'],
                'USER': db_settings['username'],
                'PASSWORD': db_settings['password'],
                'HOST': db_settings['host'],
                'PORT': db_settings['port'],
            },
        }
```

We normally place import statements at the beginning of a module, but in this case, it's helpful to keep all the remote-specific settings in one section. Here we import `Config` from `platformshconfig` ❶, which helps determine settings on the remote server. We only modify settings if the method `config.is_valid_platform()` returns `True` ❷, indicating the settings are being used on a Platform.sh server.

We modify `ALLOWED_HOSTS` to allow the project to be served by hosts ending in `.platformsh.site` ❸. All projects deployed to the free tier will be served using this host. If settings are being loaded in the deployed app's directory ❹, we set `STATIC_ROOT` so that static files are served correctly. We also set a more secure `SECRET_KEY` on the remote server ❺.

Finally, we configure the production database ❻. This is only set if the build process has finished running and the project is being served. Everything you see here is necessary to let Django talk to the Postgres server that Platform.sh set up for the project.

Using Git to Track the Project's Files

As discussed in Chapter 17, Git is a version control program that allows you to take a snapshot of the code in your project each time you implement a new feature successfully. If anything goes wrong, you can easily return to